

Competitive Exam Practice 5 — Answer Key

Queues, Circular Buffers, Deques, and Priority Queues

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

September 17, 2026

- Q1 (A) (ii) and (iii) are true.** (i) is false: stacks are LIFO, so FIFO computations are *not* efficiently supported by them. (ii) is true: linked lists give $O(1)$ insert/delete at a known position versus $O(n)$ shifting on arrays for almost all basic LIST operations. (iii) is true: wraparound reuse keeps every operation $O(1)$ worst case, while the linear two-index version strands free slots (false full) unless it pays $O(n)$ shifting. (iv) is false: queues are FIFO, so LIFO computations are *not* efficiently supported by them. Stem from GATEOverflow V2 p.240; options and answer (a) confirmed against the BYJUS GATE bank and ExamSIDE's GATE-1996 listing (2026-09-17).
- Q2 (A) full: $(\text{REAR}+1) \bmod n == \text{FRONT}$; empty: $\text{REAR} == \text{FRONT}$.** One slot is sacrificed so the two states are distinguishable: with $n - 1$ live items REAR sits one slot behind FRONT (full test fires), while the untouched initial state $\text{REAR} = \text{FRONT} = 0$ means empty. (B) fails at the start ($\text{FRONT}+1 \neq \text{REAR}$ on an empty queue); (C) swaps the two tests (initial state would read “full”); (D) misplaces the $+1$ on FRONT . Confirmed against the GATEOverflow page (accepted reasoning), Testbook's GATE-2012 solution (“option 1”), and ExamSIDE's GATE-2012 listing (2026-09-17).
- Q3 (A) Both operations can be performed in $O(1)$ time.** A circular-array queue does one array access plus one modular advance per operation — $O(1)$ worst case for both ENQUEUE and DEQUEUE, with no shifting ever. (B) describes the *linear*-array failure mode (one end drifts into $O(n)$ shifting), not the efficient implementation the stem asks for; (C)/(D) overstate both bills. Stem and options confirmed on appliedroots' GATE-2016-Set-1 listing, the SamagraCS solved page, and ExamSIDE's topic listing; answer (A) per the official GATE 2016 key and the deck's circular implementation (2026-09-17).
- Q4 (D) strictly decreasing order.** DELETETMIN always returns the smallest key present, but a stack must return the *most recently* pushed item — so each new key must be smaller than every key already in the queue, i.e. strictly decreasing. Increasing/non-decreasing would emulate a queue (FIFO), and non-increasing admits equal keys, which lets an older item with the same key come out first. Tier-1 stem and options from GATEOverflow V2 p.239, year listed on ExamSIDE under GATE CSE 1997, principle cross-checked against the documented monotonic-key construction for stack-from-priority-queue (2026-09-17).
- Q5 (B) Reverses the order of the elements in the queue.** Each frame deletes the front element, recurses on the rest, then inserts its element at the rear on the way back out — so the first-deleted (front) element is re-inserted last and ends up at the rear: the queue comes back reversed. (A) would need insert-before-recurse; (C) describes a single rotate, not a full

recursion; (D) would need the inserts to be dropped. Confirmed on the GATEOverflow page (best answer B, 56 votes), the GFG GATE-IT-2007 quiz, and ExamSIDE's listing (2026-09-17).

Q6 (A) (5, 3, 5). **dequeue:** $\text{front} = (4 + 1) \bmod 6 = 5$, $\text{count} = 4$ (live items now at indices 5, 0, 1, 2). **enqueue:** $\text{rear} = (2 + 1) \bmod 6 = 3$, $\text{count} = 5$ (items at 5, 0, 1, 2, 3). Not full ($5 < 6$). (B) miscounts **count** (a dequeue and an enqueue net to zero); (C) forgets to advance **front**; (D) forgets to advance **rear**. Confirmed by execution (2026-09-17).

Q7 (B) Output-restricted deque; dequeueRear is forbidden. The restricted cousin that closes the *deletion* end is the output-restricted deque (insert either end, remove one end only) — here only front deletions are allowed, so rear deletion is the forbidden operation. (A)/(D) misname the variant (input-restricted closes an *insertion* end, and both insertions are open here); (C) names the wrong variant even though it points at the right forbidden end. Cross-checked against the deck's restricted-cousin definitions and the BU Lecture-5 operation sets (2026-09-17).

Q8 (A) Unordered array. Price the workload: unordered pays $\approx 10,000 \times O(1)$ inserts plus $5 \times O(n)$ scans ($\sim 60,000$ elementary steps); ordered pays $\approx 10,000 \times O(n)$ shifting inserts ($\sim 10^8$) to buy 5 cheap removals — paying the linear bill on the hot operation. (B)/(D) put the linear cost where the volume is; (C) ignores that the two bills scale with different operation counts. Confirmed by execution (2026-09-17).